강화학습 교육 2일차 보고서, 김도훈

코드를 구현하는 부분에서는 PPT 자료에서 Q-Learning의 학습과 평가하는 코드를 구현할 수 있다. 하지만 그래프 출력, 격자 출력, 히트맵 출력 등은 Chat GPT를 사용하여 구현하였고 그 시간을 활용해 관련 이론들을 학습하는데 더 집중했다.

1. MDP 요소

S(상태) : 4 * 4 FrozneLake 환경

A(행동) : 왼쪽, 아래, 오른쪽, 위

P(전이 확률) : 결정적 상태 전이

R(보상) : 목표 도착 1, 나머지 0

할인율 : 0.95

강화학습 환경 생성 코드

```
env = gym.make(
"FrozenLake-v1",
map_name="4x4",
is_slippery=False # 상태 전이 확률 1
)
```

위 코드는 강화학습 환경을 설정한다. FrozenLake 환경, 4 * 4맵, 미끄러지지 않도록 설정한다. S(상태)는 FrozenLake 환경 4 * 4 환경으로 설정되었고 Is_slippery=False로 인해 P(전이 확률)은 1로 결정적 상태 전이로 설정되었다.

정책 함수

```
 # Policy 함수(앱실론-그리디 정책)
def choose_action(state, epsilon, rng):
# 탐험: 무작위 행동 선택
if rng.random() < epsilon:
```

```
return int(rng.integers(4))

# 활용: 최대 Q-value 행동 선택
best = np.flatnonzero(Q[state] == Q[state].max())

# 동률인 경우 무작위 선택
return int(rng.choice(best))
```

위이 코드는 앱실론-그리디 정책 함수이다. A(행동)은 왼쪽, 아래, 오른쪽, 위 총 네가지로 한 Step마다 아래 앱실론-그리디 정책 함수로 정해진다. 생성한 난수가 앱실론 보다 작다면 네가지 행동 중 하나를 무작위로 행동하고 앱실론 보다 크다면 Q-value가 가장 큰 값을 행동한다.

R(보상)은 목표 도착 1 외 나머지는 모두 0으로 설정하였다. 할인율은 gamma = 0.95로 설정하였다.

2. Return과 Q-Value(state-action value function)

Return(Gt)는 미래에 얻게될 할인 누적 보상이다. 강화학습 환경에서 보상은 1(도착), 0(도착 아님)이다. 그러므로 초기에 이동했을 때는 보상이 0이지만 나중에 이동했을 땐 총 보상이 1이 될 수 있다. 그러므로 return을 사용한다.

Q-Value는 특정 상태에서 특정 행동을 취했을 때 앞으로 얻을 수 있는 보상들의 총합에 대한 기대값입니다. 현재 맵이 4 * 4이고 총 상태가 16개, 행동은 위, 아래, 오른쪽, 왼쪽 총 4개이다. 그러므로 총 Q-Value는 64개이다. 그러므로 Q-table이 필요하다.

```
Q = np.zeros((
env.observation_space.n,
env.action_space.n # 0:왼쪽, 1:아래쪽, 2:오른쪽, 3:위쪽
))
```

위 코드는 16 * 4 Q-table를 생성하는 코드이다. 초기 Q-table의 요소는 모두 0이다.

3. Q-Learning

```
#학습률
alpha = 0.1
#할인율
gamma = 0.95
#에피소드
episodes = 5000
# Q-learning 학습
for episode in range(episodes): #에피소드 반복
# 매에피소드마다 0으로 상태 초기화
state, _ = env.reset(seed=0 if episode == 0 else None)
epsilon = max(0.05, 0.999 ** episode) # 에피소드가 증가할수록 앱실론 감소
total = 0
while True:
#행동 선택(앱실론-그리디 정책)
action = choose_action(state, epsilon, rng)
#보상 R
ns, r, terminated, truncated, _ = env.step(action)
# Q-learning target 계산
if terminated: #에피소드가 종료되었을 때
target = r
else: #에피소드가 종료되지 않았을 때
target = r + gamma * Q[ns].max() # 미래 가지 고려 계산
# Q-value 업데이트
Q[state, action] += alpha * (target - Q[state, action])
total += r #보상 단순 합계
state = ns
if terminated or truncated: #종료되었다면 by 구멍, 시간
break
returns.append(total)
train_successes.append(int(total == 1))
```

Q-Learning 업데이트 수식

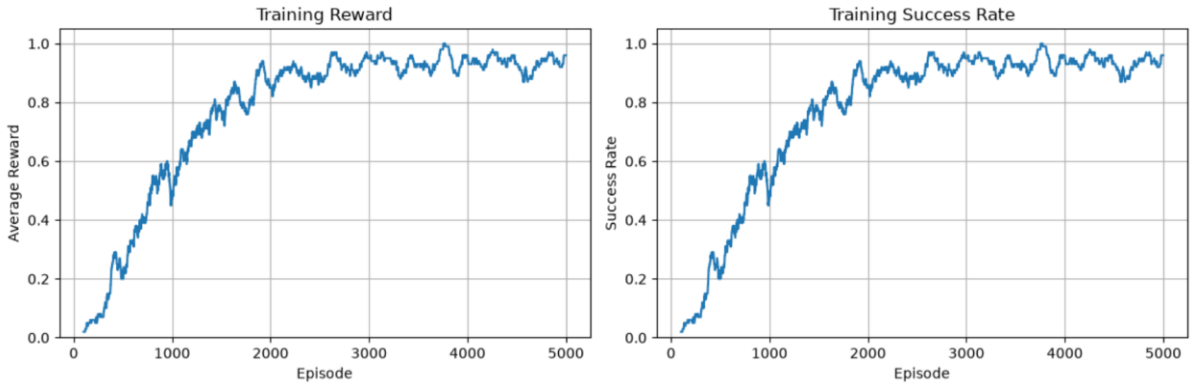
$$Q(s,\ a)\ \leftarrow\ Q(s,\ a)\ +\ \alpha[r\ +\ \gamma\max Q(\ s',\ a')\ -\ Q(s,\ a)]$$

실제 코드에서도 위와 같은 수식을 구현하였다. 코드에서는 에피소드의 종료 여부에 따라 target 값을 나누었다. 에피소드가 구멍에 빠지거나 시간에 의해 종료된다면 단순히 target = r 이고 그게 아니면 위의 수식을 사용해 target 값을 구한다.

예를 들면 에이전트가 상태 10에서 아래쪽 행동을 선택하여 상태 14로 상황을 가정해보자. 코드에서 목표 도착 외이면 보상은 0이고 기존의 Q-Value는 0.2 다음 상태의 최대 Q-value는 0.8이라고 하자. 먼저 target 값을 계산해보면 0 + 0.95 * 0.8 = 0.76이고 전체 계산은 0.2 + 0.1 * (0.76 - 0.2) = 0.256이 된다. 기존 Q-Value는 0.2에서 0.256으로 증가했다.

이는 현재 행동의 보상이 0이더라도 미래에 얻을 수 있는 보상이 현재 행동의 가치에 반영될 수 있음을 보여준다.

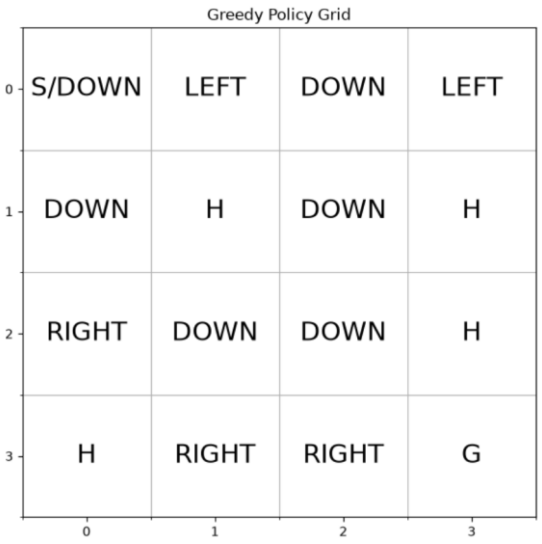
4. 결과 분석

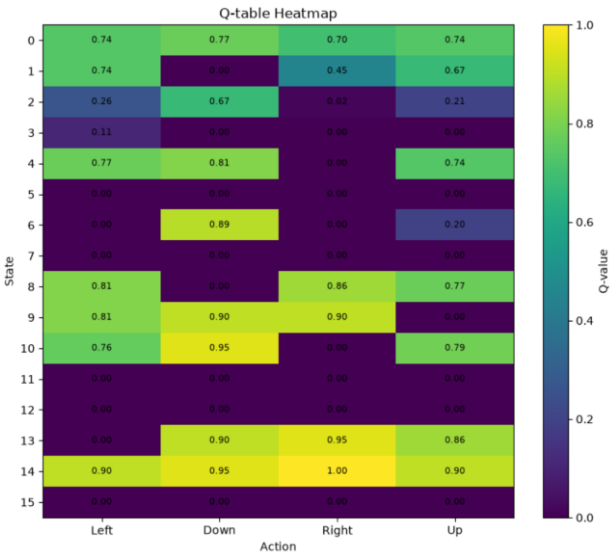


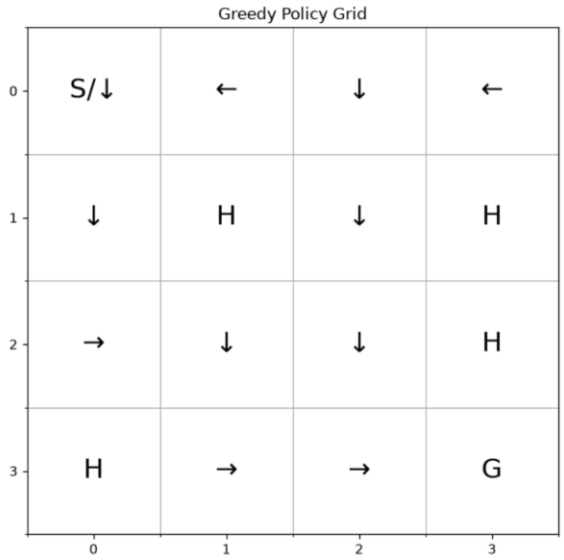
위 두 그래프는 5000번의 에피소드 동안 학습 성능이 어떻게 변화했는지 나타낸다. 학습 초기에는 보상과 성공률이 높지 않았다. 약 2000번째 에피소드부터 성공율이 80%를 안정적으로 넘고 있다. 이러한 결과는 Q-Learning 업데이트를 통해 Q-Value가 학습되었기 때문으로 판단된다. 초기에는 Q-table의 모든 요소들이 0이므로 어떤 행동이 유리한지 판단하기 어렵다. 하지만 학습이 반복되면서 점차 목표에 도달하는 경로를 학습하게 된다.

또한 앱실론-그리디 정책 역시 성공률에 영향을 미쳤을 것이다. 학습 초기에는 앱실론이

1.0이다. 그래서 무작위 행동을 통해 탐험을 하지만 Q-Learning이 진행되면서 앱실론은 감소하고 Q-Value가 높은 행동을 활용하는 비율을 증가한다. 그러므로 후반에 성공률이 높았을 것으로 판단한다.







오른쪽 사진은 Q-table을 바탕으로 각 상태에서 Q-Value가 가장 높은 핼동을 화살표로 나타낸 결과이다. 즉 최단 경로라고 생각할 수 있다.

왼쪽 사진은 Q-table을 시각화한 결과이다. 세로는 0부터 15 총 16개의 상태를 나타내고 가로는 왼쪽, 아래쪽, 오른쪽, 위쪽 네 가지 행동을 나타낸다.

평가 결과

Total Episodes: 200

Success Count: 200

Success Rate: 100.00%

Average Successful Path Length: 6.00

Min Successful Path Length: 6

Max Successful Path Length: 6

Successful Path Length Distribution: {6: 200}

위 결과는 학습된 Q-table을 이용하여 greedy 정책 a = argmax Q(s,a) 을 선택했을 때 평가 결과이다. 학습할 땐 앱실론-greedy 정책을 사용하여 무작위로 탐험했지만 평가에서는 앱실론을 0으로 설정하여 Q-value가 최대인 행동을 선택하도록 했다. 그리고 평균, 최소, 최대 STEP 모두 6 STEP으로 출발점에서 목표점까지 이동했다. 200번 동안 200번 모두 성공하여 성공률 또한 100%가 나왔다. 이러한 결과는 4 by 4 환경에서 모두 최단 거리로 목표 지점까지 이동했다는 것이고 Q-Learning을 통해 최단 경로를 선택하는 정책(policy)이 형성되었음을 확인할 수 있다. 하지만 본 코드는 is_slippery = False로 설정된 결정적 상태 전이 환경에서 진행되었다. 그래서 확률적 상태 전이가 발생하는 환경에서는 동일한 성공률을 보장한다고 볼 수 없다.